

Noble Funded - Technical Requirements

- [NOBLE FUNDED - COMPLETE TECHNICAL REQUIREMENTS DOCUMENT](#)
 - [TABLE OF CONTENTS](#)
 - [1. PROJECT OVERVIEW](#)
 - [1.1 Business Model](#)
 - [1.2 Core Workflow](#)
 - [1.3 Technology Stack \(Proposed\)](#)
 - [2. SYSTEM ARCHITECTURE](#)
 - [2.1 High-Level Architecture](#)
 - [2.2 Database Architecture](#)
 - [3. USER ROLES & PERMISSIONS](#)
 - [3.1 Role Hierarchy](#)
 - [3.2 Permission Matrix](#)
 - [4. FRONTEND REQUIREMENTS](#)
 - [4.1 Public Website Pages](#)
 - [4.2 Trader Dashboard \(Customer-Facing\)](#)
 - [4.3 Mobile Responsiveness](#)
 - [5. BACKEND REQUIREMENTS](#)
 - [5.1 API Endpoints Structure](#)
 - [5.2 Core API Endpoints](#)
 - [5.3 Real-Time Monitoring System](#)
 - [5.4 Error Handling](#)
 - [6. DATABASE SCHEMA](#)
 - [6.1 Users Table](#)
 - [6.2 Challenges Table](#)
 - [6.3 Transactions Table](#)
 - [6.4 Payouts Table](#)
 - [6.5 Team Members Table](#)
 - [6.6 Activity Logs Table](#)
 - [6.7 System Settings Table](#)
 - [7. ADMIN DASHBOARD SPECIFICATIONS](#)
 - [7.1 Dashboard Routes](#)
 - [7.2 Super Admin Dashboard](#)
 - [7.3 Compliance Dashboard](#)
 - [7.4 Support Dashboard](#)
 - [7.5 Marketing Dashboard](#)
 - [7.6 Developer Dashboard](#)
 - [8. ROLE-BASED ACCESS CONTROL \(RBAC\)](#)
 - [8.1 Implementation Requirements](#)
 - [8.2 Frontend Route Protection](#)
 - [8.3 UI Element Hiding](#)
 - [9. PAYMENT INTEGRATION \(FLUTTERWAVE\)](#)
 - [9.1 Payment Flow](#)
 - [9.2 Flutterwave Integration Endpoints](#)
 - [9.3 Supported Payment Methods](#)
 - [10. MT5 INTEGRATION](#)
 - [10.1 Broker API Requirements](#)
 - [10.2 Integration Points](#)
 - [10.3 Error Handling](#)
 - [11. REAL-TIME MONITORING SYSTEM](#)
 - [11.1 Architecture](#)
 - [11.2 Implementation Details](#)
 - [11.3 Alerting System](#)
 - [11.4 Performance Requirements](#)
 - [12. SECURITY REQUIREMENTS](#)
 - [12.1 Authentication & Authorization](#)
 - [12.2 API Security](#)
 - [12.3 Data Protection](#)
 - [12.4 Input Validation](#)
 - [12.5 Audit Logging](#)
 - [13. DEPLOYMENT & INFRASTRUCTURE](#)
 - [13.1 Hosting Setup](#)
 - [13.2 Environment Variables](#)
 - [13.3 Domain Setup](#)
 - [13.4 Monitoring & Logging](#)

- [13.5 Backup Strategy](#)
- [14. TESTING REQUIREMENTS](#)
 - [14.1 Unit Tests](#)
 - [14.2 Integration Tests](#)
 - [14.3 Manual Testing Checklist](#)
- [15. TIMELINE & MILESTONES](#)
 - [15.1 Development Phases](#)
 - [15.2 Milestone Checklist](#)
- [16. DELIVERABLES](#)
 - [16.1 What Developer Must Provide](#)
 - [16.2 Ongoing Support](#)
- [17. COMMUNICATION & COLLABORATION](#)
 - [17.1 Development Workflow](#)
 - [17.2 Project Management](#)
- [18. ACCEPTANCE CRITERIA](#)
 - [18.1 Definition of Done](#)
 - [18.2 Launch Criteria](#)
- [19. QUESTIONS FOR DEVELOPER](#)
- [20. APPENDIX](#)
 - [20.1 Glossary](#)
 - [20.2 Reference Materials](#)
 - [20.3 Contact Information](#)
- [SIGNATURE](#)

NOBLE FUNDED - COMPLETE TECHNICAL REQUIREMENTS DOCUMENT

Version: 1.0

Date: February 20, 2026

Project: Noble Funded Prop Trading Platform

Client: [Your Name]

Developer: [Developer Name]

TABLE OF CONTENTS

1. Project Overview
2. System Architecture
3. User Roles & Permissions
4. Frontend Requirements
5. Backend Requirements
6. Database Schema
7. Admin Dashboard Specifications
8. Role-Based Access Control
9. Payment Integration
10. MT5 Integration
11. Real-Time Monitoring System
12. Security Requirements
13. Deployment & Infrastructure
14. Testing Requirements
15. Timeline & Milestones

1. PROJECT OVERVIEW

1.1 Business Model

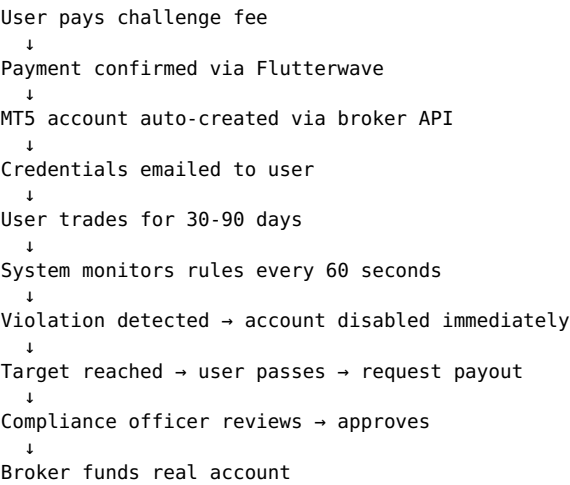
Noble Funded is a prop trading firm platform targeting the Nigerian market with two account types:

Naira Accounts: - Entry fee: ₦10,000 - ₦50,000 - Trading capital: ₦200,000 - ₦1,000,000 - Profit target: 200% of starting balance - Max drawdown: 10% - Duration: 60 days

Dollar Accounts: - Entry fee: ₦100,000 - ₦450,000 - Trading capital: \$15,000 - \$100,000 - Profit target: 100% of starting balance (2-phase evaluation) - Max drawdown: 5% - Duration: 90 days

Profit Split: 80% trader, 20% company

1.2 Core Workflow



1.3 Technology Stack (Proposed)

Frontend: - Framework: React.js or Next.js - Styling: Tailwind CSS - Hosting: Netlify - Build tool: Vite or Next.js built-in

Backend: - Framework: Node.js (Express) or Python (FastAPI/Django) - Hosting: Render - Real-time: WebSockets or Server-Sent Events

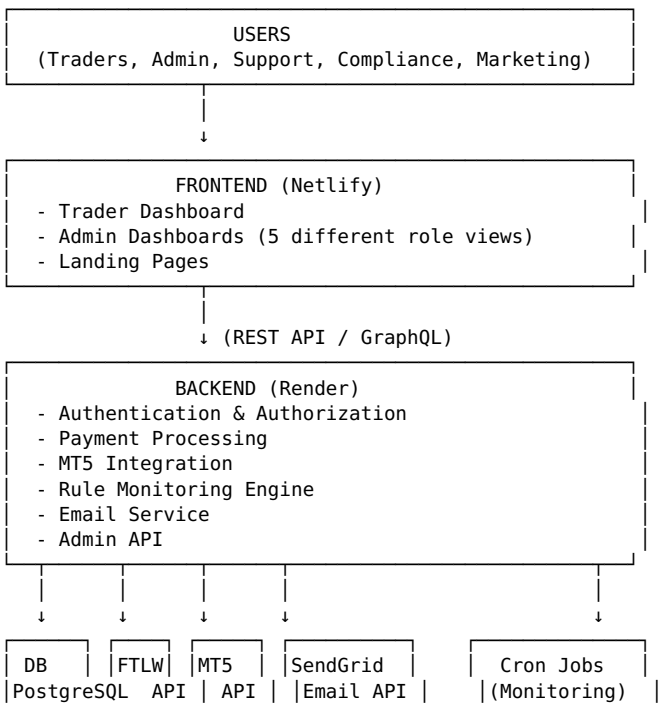
Database: - Primary: PostgreSQL (Render managed) - Caching: Redis (optional for Phase 2)

External Services: - Payment: Flutterwave API - Email: SendGrid or Resend - MT5: Broker’s API (to be provided) - File storage: Cloudinary or AWS S3

Version Control: - GitHub repository (company-owned) - Daily commits required

2. SYSTEM ARCHITECTURE

2.1 High-Level Architecture



2.2 Database Architecture

Core Tables: - users - challenges - mt5_accounts - transactions - payouts - team_members - activity_logs - system_settings

(Detailed schema in Section 6)

3. USER ROLES & PERMISSIONS

3.1 Role Hierarchy

1. Super Admin (Founder)

Full access to everything
2. Compliance Officer

Challenge monitoring, payout approval, user management
3. Customer Support

User assistance, password resets, ticket management
4. Marketing Team

Analytics viewing only (read-only)
5. Developer

Technical backend access only
6. Trader (Customer)

Their own dashboard only

3.2 Permission Matrix

Feature	Super Admin	Compliance	Support	Marketing	Developer	Trader
View all users	✓	✓	✓	✗	✗	✗
View user trades	✓	✓	✗	✗	✗	Own only
Approve payouts	✓	✓	✗	✗	✗	✗
Ban users	✓	✓	✗	✗	✗	✗
Reset passwords	✓	✓	✓	✗	✗	Own only
View financial data	✓	✗	✗	✗	✗	✗
Change system settings	✓	✗	✗	✗	✗	✗
Add team members	✓	✗	✗	✗	✗	✗
View analytics	✓	✓	✗	✓	✗	✗
Access server logs	✓	✗	✗	✗	✓	✗
Deploy code	✓	✗	✗	✗	✓	✗
Create challenges	✓	✓	✓	✗	✗	Buy only
View support tickets	✓	✓	✓	✗	✗	Own only

4. FRONTEND REQUIREMENTS

4.1 Public Website Pages

Required Pages:

- Homepage**
 - Hero section with value proposition
 - Pricing comparison (Naira vs Dollar accounts)
 - Features overview
 - Testimonials (placeholder for now)
 - FAQ section
 - CTA buttons
- Pricing Page**
 - Side-by-side comparison table
 - Clear rules for each account type
 - "Buy Challenge" buttons linking to checkout
- How It Works**
 - Step-by-step process
 - Visual illustrations
 - Video embed (optional)
- FAQ Page**
 - Expandable sections
 - Search functionality
- Contact Page**
 - Contact form
 - Email: contact@noblefunded.com
 - WhatsApp link
 - Social media links
- Login/Registration Pages**
 - Email + Password authentication
 - "Forgot Password" flow
 - Social login (optional Phase 2)

4.2 Trader Dashboard (Customer-Facing)

Layout:

HEADER: Logo Dashboard Challenges Profile
SIDEBAR (Mobile: Bottom Nav) - Overview - Active Challenges - Challenge History - Payouts - Support - Settings
MAIN CONTENT AREA (Dynamic based on selected page)

Dashboard Sections:

Overview Page: - Active challenges count - Total profit/loss - Upcoming payouts - Quick stats - Recent activity

Active Challenges Page:

For each active challenge, display:

Challenge #12345 - Naira ₦200,000
Status: ACTIVE ✓ Days Remaining: 23/60 Progress: 43% to target
Starting Balance: ₦200,000 Current Balance: ₦286,000 Current Equity: ₦289,450 Profit: +₦86,000 (43%)

Target: ₦400,000 (200%)
RULES COMPLIANCE: Max Drawdown: 3.2% / 10% ✓ Daily Loss: 1.5% / 3% ✓
MT5 Login: 12345678 Server: Broker-Demo Password: ***** [Show] [Copy]
[View Detailed Stats] [Download MT5]

Key Features: - Real-time balance updates (refresh every 60 seconds) - Visual progress bars for profit target - Color-coded rule compliance (green = safe, yellow = warning, red = danger) - Countdown timer for days remaining - Chart showing equity curve over time - Trade history table (last 20 trades)

Challenge History Page: - List of all past challenges - Status: Passed ✓, Failed ✗, In Progress ⌘ - Filters: All, Passed, Failed, Active - Sort by: Date, Status, Profit

Payouts Page: - Request payout button (only shows when eligible) - Payout history table - Status: Pending, Approved, Rejected, Paid - Amount, Date, Notes

Support Page: - Ticket creation form - Ticket history - Status tracking - Live chat widget (optional Phase 2)

Settings Page: - Profile information - Change password - Email preferences - Two-factor authentication (Phase 2)

4.3 Mobile Responsiveness

CRITICAL: All pages must be fully responsive

Breakpoints: - Mobile: < 768px - Tablet: 768px - 1024px - Desktop: > 1024px

Mobile-specific requirements: - Touch-friendly buttons (min 44px height) - Readable text (min 16px font size) - No horizontal scrolling - Fast loading on slow connections - Works on iOS Safari and Chrome Android

5. BACKEND REQUIREMENTS

5.1 API Endpoints Structure

Base URL: https://api.noblefunded.com OR https://noble-funded-api.onrender.com

Authentication: - JWT (JSON Web Tokens) - Access token (15 min expiry) - Refresh token (7 days expiry) - Stored in httpOnly cookies

5.2 Core API Endpoints

Authentication Endpoints

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
POST /api/auth/refresh-token
POST /api/auth/forgot-password
POST /api/auth/reset-password/:token
GET /api/auth/verify-email/:token
```

User Endpoints

```
GET /api/users/me (Get current user info)
PUT /api/users/me (Update profile)
PUT /api/users/me/password (Change password)
GET /api/users/:id (Admin only)
GET /api/users (Admin only - list all)
PUT /api/users/:id/ban (Admin/Compliance only)
```

Challenge Endpoints

GET	/api/challenges	(User's own challenges)
GET	/api/challenges/:id	(Challenge details)
POST	/api/challenges/buy	(Purchase new challenge)
GET	/api/challenges/:id/stats	(Real-time stats from MT5)

Payment Endpoints

POST	/api/payments/initiate	(Start payment flow)
POST	/api/payments/webhook	(Flutterwave webhook)
GET	/api/payments/:id	(Payment details)
GET	/api/payments/history	(User's payment history)

Payout Endpoints

POST	/api/payouts/request	(Request payout)
GET	/api/payouts	(User's payout history)
GET	/api/payouts/:id	(Payout details)
PUT	/api/payouts/:id/approve	(Compliance only)
PUT	/api/payouts/:id/reject	(Compliance only)

MT5 Endpoints

POST	/api/mt5/create-account	(Create MT5 account)
GET	/api/mt5/:login/balance	(Get current balance)
GET	/api/mt5/:login/trades	(Get trade history)
GET	/api/mt5/:login/equity	(Get current equity)
POST	/api/mt5/:login/disable	(Disable account)

Admin Endpoints

GET	/api/admin/dashboard/stats	(Overview statistics)
GET	/api/admin/users	(All users with filters)
GET	/api/admin/challenges	(All challenges)
GET	/api/admin/revenue	(Financial reports)
GET	/api/admin/activity-log	(System activity)
POST	/api/admin/team/add	(Add team member)
DELETE	/api/admin/team/:id	(Remove team member)

Support Endpoints

POST	/api/support/tickets	(Create ticket)
GET	/api/support/tickets	(List tickets)
GET	/api/support/tickets/:id	(Ticket details)
PUT	/api/support/tickets/:id	(Update ticket)
POST	/api/support/tickets/:id/reply	(Add reply)

5.3 Real-Time Monitoring System

CRITICAL REQUIREMENT: This system MUST run 24/7 without fail

Purpose: Monitor all active challenges for rule violations and disable accounts immediately when violations occur.

How It Works:

```
// Pseudocode example

setInterval(async () => {
  // Run every 60 seconds
  const activeChallenges = await db.getChallenges({ status: 'active' });

  for (const challenge of activeChallenges) {
    try {
      // Get live data from MT5
      const mt5Data = await brokerAPI.getAccountData(challenge.mt5_login);

      // Check all rules
      const violations = checkRules(challenge, mt5Data);
```

```

    if (violations.length > 0) {
      // Immediate action required

      // 1. Disable MT5 account
      await brokerAPI.disableAccount(challenge.mt5_login);

      // 2. Update database
      await db.updateChallenge(challenge.id, {
        status: 'failed',
        failed_reason: violations[0],
        failed_at: new Date(),
        final_balance: mt5Data.balance,
        final_equity: mt5Data.equity
      });

      // 3. Send email notification
      await emailService.send({
        to: challenge.user_email,
        subject: 'Challenge Failed - Account Disabled',
        body: `Your challenge has been disabled due to: ${violations[0]}`
      });

      // 4. Log activity
      await db.logActivity({
        type: 'challenge_failed',
        challenge_id: challenge.id,
        reason: violations[0],
        mt5_data: mt5Data
      });
    }

    // Check if passed
    if (hasPassedChallenge(challenge, mt5Data)) {
      await db.updateChallenge(challenge.id, {
        status: 'passed',
        passed_at: new Date()
      });

      await emailService.send({
        to: challenge.user_email,
        subject: 'Congratulations! You Passed! 🎉',
        body: 'You can now request payout...'
      });
    }

  } catch (error) {
    // Log error but continue monitoring other accounts
    console.error(`Error monitoring challenge ${challenge.id}:`, error);
    await db.logError({
      challenge_id: challenge.id,
      error: error.message,
      timestamp: new Date()
    });
  }
}

}, 60000); // Every 60 seconds

```

Rule Checking Function:

```

function checkRules(challenge, mt5Data) {
  const violations = [];

  // 1. Max Drawdown Check
  const drawdown = (challenge.starting_balance - mt5Data.equity) / challenge.starting_balance;
  const maxDrawdown = challenge.account_type === 'naira' ? 0.10 : 0.05;

  if (drawdown >= maxDrawdown) {
    violations.push(`Max drawdown exceeded: ${(drawdown * 100).toFixed(2)}%`);
  }

  // 2. Daily Loss Check
  const todayLoss = calculateDailyLoss(challenge.mt5_login);
  const maxDailyLoss = challenge.account_type === 'naira' ? 0.03 : 0.03;

  if (todayLoss >= maxDailyLoss) {

```

```

    violations.push(`Daily loss limit exceeded: ${((todayLoss * 100).toFixed(2))}%`);
  }

  // 3. Profit Target Check
  const profit = (mt5Data.equity - challenge.starting_balance) / challenge.starting_balance;
  const target = challenge.account_type === 'naira' ? 2.0 : 1.0;

  if (profit >= target) {
    // This is good! User passed
    return [];
  }

  // 4. Time Limit Check
  const daysElapsed = daysSince(challenge.created_at);
  const maxDays = challenge.account_type === 'naira' ? 60 : 90;

  if (daysElapsed >= maxDays && profit < target) {
    violations.push('Time limit reached without hitting target');
  }

  return violations;
}

```

Monitoring Requirements:

1. **Reliability:** Must run 24/7 even if main API goes down
2. **Error Handling:** If one check fails, continue with others
3. **Alerting:** If monitoring stops, alert admin immediately
4. **Logging:** Log every check result (for audit trail)
5. **Performance:** Must handle 1000+ concurrent challenges
6. **Redundancy:** If primary monitoring fails, backup system activates

Implementation Options:

Option A: Cron Job (Recommended) - Node.js cron package - Runs on Render background worker - Separate from main API process

Option B: Queue System - Use Bull or BullMQ - Redis-backed job queue - More robust for scale

5.4 Error Handling

All API endpoints must return consistent error format:

```

{
  "success": false,
  "error": {
    "code": "PAYMENT_FAILED",
    "message": "Payment could not be processed",
    "details": "Insufficient funds"
  }
}

```

Common Error Codes: - AUTH_REQUIRED: User not logged in - FORBIDDEN: User lacks permission - NOT_FOUND: Resource doesn't exist - VALIDATION_ERROR: Invalid input data - PAYMENT_FAILED: Payment processing error - MT5_ERROR: MT5 API error - INTERNAL_ERROR: Server error

6. DATABASE SCHEMA

6.1 Users Table

```

CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  full_name   VARCHAR(255) NOT NULL,
  phone       VARCHAR(20),
  country     VARCHAR(2) DEFAULT 'NG',
  role        VARCHAR(20) DEFAULT 'trader',
  status      VARCHAR(20) DEFAULT 'active',
  email_verified BOOLEAN DEFAULT false,
  kyc_status  VARCHAR(20) DEFAULT 'pending',

```

```

        created_at      TIMESTAMP DEFAULT NOW(),
        updated_at      TIMESTAMP DEFAULT NOW(),
        last_login      TIMESTAMP
    );

-- Indexes
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role ON users(role);
CREATE INDEX idx_users_status ON users(status);

```

6.2 Challenges Table

```

CREATE TABLE challenges (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id           UUID REFERENCES users(id),
    account_type      VARCHAR(10) NOT NULL, -- 'naira' or 'dollar'
    starting_balance  DECIMAL(12,2) NOT NULL,
    current_balance   DECIMAL(12,2) NOT NULL,
    current_equity     DECIMAL(12,2) NOT NULL,
    profit_target     DECIMAL(12,2) NOT NULL,
    max_drawdown_pct  DECIMAL(5,2) NOT NULL,
    max_daily_loss_pct DECIMAL(5,2) NOT NULL,
    duration_days     INTEGER NOT NULL,
    status            VARCHAR(20) DEFAULT 'pending', -- pending, active, passed, failed
    mt5_login         VARCHAR(50),
    mt5_password      VARCHAR(50),
    mt5_server        VARCHAR(100),
    phase            INTEGER DEFAULT 1, -- For dollar accounts (Phase 1, Phase 2)
    started_at        TIMESTAMP,
    passed_at         TIMESTAMP,
    failed_at         TIMESTAMP,
    failed_reason     TEXT,
    created_at        TIMESTAMP DEFAULT NOW(),
    updated_at        TIMESTAMP DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_challenges_user ON challenges(user_id);
CREATE INDEX idx_challenges_status ON challenges(status);
CREATE INDEX idx_challenges_mt5 ON challenges(mt5_login);

```

6.3 Transactions Table

```

CREATE TABLE transactions (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id           UUID REFERENCES users(id),
    challenge_id      UUID REFERENCES challenges(id),
    type             VARCHAR(20) NOT NULL, -- 'challenge_purchase', 'payout'
    amount           DECIMAL(12,2) NOT NULL,
    currency         VARCHAR(3) NOT NULL, -- 'NGN', 'USD'
    status           VARCHAR(20) DEFAULT 'pending', -- pending, completed, failed
    payment_method   VARCHAR(50), -- 'card', 'bank_transfer', 'usdt'
    payment_provider VARCHAR(50), -- 'flutterwave'
    provider_ref     VARCHAR(100),
    metadata         JSONB,
    created_at       TIMESTAMP DEFAULT NOW(),
    updated_at       TIMESTAMP DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_transactions_user ON transactions(user_id);
CREATE INDEX idx_transactions_status ON transactions(status);
CREATE INDEX idx_transactions_ref ON transactions(provider_ref);

```

6.4 Payouts Table

```

CREATE TABLE payouts (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id           UUID REFERENCES users(id),
    challenge_id      UUID REFERENCES challenges(id),
    amount           DECIMAL(12,2) NOT NULL,
    currency         VARCHAR(3) NOT NULL,
    status           VARCHAR(20) DEFAULT 'pending', -- pending, approved, rejected, paid
    payout_method    VARCHAR(50), -- 'bank_transfer', 'usdt'
    bank_name        VARCHAR(100),

```

```

account_number    VARCHAR(50),
account_name      VARCHAR(100),
usdt_address      VARCHAR(100),
approved_by      UUID REFERENCES users(id),
approved_at       TIMESTAMP,
rejected_reason    TEXT,
paid_at           TIMESTAMP,
notes            TEXT,
created_at        TIMESTAMP DEFAULT NOW(),
updated_at        TIMESTAMP DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_payouts_user ON payouts(user_id);
CREATE INDEX idx_payouts_status ON payouts(status);
CREATE INDEX idx_payouts_challenge ON payouts(challenge_id);

```

6.5 Team Members Table

```

CREATE TABLE team_members (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id),
  role        VARCHAR(50) NOT NULL, -- 'super_admin', 'compliance', 'support', 'marketing',
'developer'
  permissions JSONB NOT NULL,
  added_by    UUID REFERENCES users(id),
  created_at  TIMESTAMP DEFAULT NOW(),
  updated_at  TIMESTAMP DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_team_role ON team_members(role);

```

6.6 Activity Logs Table

```

CREATE TABLE activity_logs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id),
  action       VARCHAR(100) NOT NULL, -- 'user_login', 'payout_approved', 'challenge_failed'
  resource_type VARCHAR(50), -- 'user', 'challenge', 'payout'
  resource_id  UUID,
  details      JSONB,
  ip_address   VARCHAR(45),
  user_agent   TEXT,
  created_at   TIMESTAMP DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_logs_user ON activity_logs(user_id);
CREATE INDEX idx_logs_action ON activity_logs(action);
CREATE INDEX idx_logs_created ON activity_logs(created_at);

```

6.7 System Settings Table

```

CREATE TABLE system_settings (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  key         VARCHAR(100) UNIQUE NOT NULL,
  value       TEXT NOT NULL,
  type        VARCHAR(20) NOT NULL, -- 'string', 'number', 'boolean', 'json'
  description TEXT,
  updated_by  UUID REFERENCES users(id),
  updated_at  TIMESTAMP DEFAULT NOW()
);

```

7. ADMIN DASHBOARD SPECIFICATIONS

7.1 Dashboard Routes

/admin	→ Super Admin Dashboard
/admin/compliance	→ Compliance Dashboard
/admin/support	→ Support Dashboard
/admin/marketing	→ Marketing Dashboard

7.2 Super Admin Dashboard

Route: /admin

Access: Super Admin only (Founder)

Layout:

HEADER: Noble Funded Admin [Your Name] Logout	
SIDEBAR	MAIN CONTENT
Overview	TODAY'S OVERVIEW Revenue: ₦847,000 New Signups: 23 Active Challenges: 156 Pending Payouts: 8 (₦1.2M) Failed Today: 12 REVENUE CHART (Last 30 days) [Line chart showing daily revenue] QUICK ACTIONS [View Users] [Approve Payouts] [Check Failed] [System Settings] RECENT ACTIVITY • John passed challenge #123 • Jane's payout approved (₦450k) • Mike failed max drawdown
Users	
Challenges	
Payouts	
Revenue	
Team	
Settings	
Activity	
Logs	

Dashboard Sections:

Overview Tab: - Key metrics cards - Revenue chart (daily, weekly, monthly views) - User growth chart - Challenge statistics - Recent activity feed

Users Tab:

Features: - Search bar (by email, name, ID) - Filter by: Status (active, banned), KYC status - Sort by: Joined date, Total spent, Challenges passed - Export to CSV

User table columns: - ID - Name - Email - Challenges (total/active/passed/failed) - Total Spent - Total Earned - Status - Joined Date - Actions: [View Details] [Edit] [Ban/Unban]

Challenges Tab:

Features: - Filter by: Status, Account type, Date range - Search by: User, Challenge ID, MT5 login - Export to CSV

Challenge table columns: - Challenge ID - User - Type (Naira/Dollar) - Starting Balance - Current Balance - Profit % - Status - Days Remaining - Actions: [View Details] [Disable]

Payouts Tab:

Features: - Filter by: Status (pending, approved, rejected, paid) - Sort by: Date, Amount - Bulk approve (select multiple and approve at once)

Payout table columns: - Payout ID - User - Challenge - Amount - Method (Bank/USDT) - Status - Requested Date - Actions: [Approve] [Reject] [View Details]

Revenue Tab:

Features: - Date range picker - Chart types: Line, Bar, Pie - Export reports

Metrics: - Total revenue (all time) - Revenue by period (daily, weekly, monthly, yearly) - Revenue by account type (Naira vs Dollar) - Challenge fee revenue vs payout costs - Profit margin - Average revenue per user - Revenue forecast

Team Tab:

Features: - Add new team member - View all team members - Edit permissions - Remove team members

Team table columns: - Name - Email - Role - Permissions - Added By - Added Date - Last Active -
Actions: [Edit] [Remove]

Settings Tab:

System settings: - Challenge prices (can change per account type) - Profit split percentage - Max drawdown limits - Daily loss limits - Challenge duration - Payout minimums - Email templates - Maintenance mode toggle

Activity Logs Tab:

Features: - Filter by: User, Action type, Date range - Search by: User email, Resource ID - Export logs

Log columns: - Timestamp - User - Action (e.g., “Approved payout”, “User logged in”) - Resource (e.g., Payout #123) - IP Address - Details

7.3 Compliance Dashboard

Route: /admin/compliance

Access: Compliance Officer + Super Admin

Purpose: Monitor challenges and approve payouts

Layout:

COMPLIANCE DASHBOARD [Name] Logout	
SIDEBAR	MAIN CONTENT
Overview Monitor Payouts Flagged Violations Reports	PENDING REVIEW Payout Requests: 8 Flagged Accounts: 3 Violation Reports: 5 PAYOUT APPROVAL QUEUE #1: John - ₦450,000 Challenge: ₦200k → ₦400k (200%) Status: Passed ✓ Trade Count: 87 [View Trades] [Approve] [Reject] #2: Jane - ₦280,000 ...

Key Features:

Monitor Tab: - Real-time view of all active challenges - Color-coded by risk: - 🟢 Green: Safe (< 50% drawdown limit) - 🟡 Yellow: Warning (50-80% drawdown limit) - 🔴 Red: Danger (80-95% drawdown limit) - Quick disable button - View detailed trade history

Payouts Tab: - Payout approval queue - For each payout, show: - User details - Challenge performance summary - Trade history - Screenshots/proof (if uploaded) - Approve/Reject buttons - Add notes field

Approval Flow:

Payout Request appears in queue

↓

Compliance Officer clicks "View Details"

↓

Reviews:

- Challenge stats
- Trade history
- Trading patterns (consistent or gambling?)
- No violations

↓

Decision:

- Approve → User notified → Broker funds account
- Reject → User notified with reason → Can appeal

Flagged Accounts Tab: - Accounts flagged for manual review - Reasons: Suspicious trading patterns, rapid wins, potential manipulation - Can add notes and investigation details

Violation Reports Tab: - All failed challenges - Reason for failure - Statistics on common violations - Helpful for improving rules

7.4 Support Dashboard

Route: /admin/support

Access: Support Team + Compliance + Super Admin

Purpose: Help users and manage tickets

Layout:

SUPPORT DASHBOARD [Name] Logout	
SIDEBAR	MAIN CONTENT
Tickets Users FAQ Messages	OPEN TICKETS: 12 In Progress: 8 Resolved Today: 34 [Search User] [email or name] TICKET QUEUE ● #456 - "Payment not working" User: john@email.com Created: 2 hours ago [View] [Assign to Me] ● #455 - "Can't login MT5" ...

Key Features:

User Search: - Search by email, name, phone, challenge ID, MT5 login - Shows user's full profile: - Personal info - All challenges (active, past) - Payment history - Support ticket history - Notes from previous interactions

Quick Actions: - Reset password - Resend MT5 credentials - Manually create challenge (if payment succeeded but account not created) - Add note to user profile

Ticket System:

Ticket details view:

TICKET #456 - Payment not working
Status: OPEN ●
Priority: High

User: John Doe (john@email.com) Created: 2 hours ago Last Reply: 30 minutes ago
CONVERSATION: John (2 hours ago): "I tried to buy a challenge but my card keeps getting declined. I have money in my account." [Reply box]
[Send] [Add Screenshot] [Escalate] [Mark as Resolved]
QUICK REPLIES: • Check payment method • Try different card • Contact bank for authorization
USER INFO: Challenges: 2 (1 active, 1 failed) Last Login: 3 hours ago Previous Tickets: 1 (resolved)

Support Features: - Canned responses (pre-written answers for common questions) - File attachments (screenshots, documents) - Internal notes (not visible to user) - Escalation to compliance or admin - Ticket assignment (assign to specific support agent) - Auto-close after 7 days of inactivity

7.5 Marketing Dashboard

Route: /admin/marketing

Access: Marketing Team + Super Admin

Purpose: View analytics and performance metrics

Layout:

MARKETING DASHBOARD [Name] Logout	
SIDEBAR	MAIN CONTENT
Overview Traffic Sources Conversions Campaigns Reports	THIS MONTH New Signups: 347 Purchases: 89 (25.6% conversion) Revenue: ₦8.9M Ad Spend: ₦2.1M (ROI: 4.2x) TRAFFIC SOURCES [Pie chart] • Facebook: 42% • Instagram: 28% • Twitter: 15% • Google: 10% • Direct: 5%

Key Metrics:

Overview Tab: - User acquisition funnel: - Landing page visits - Signups - Challenge purchases - Pass rate - Repeat purchases

Traffic Sources Tab: - Where users came from - Cost per acquisition by source - Best performing channels - UTM tracking

Conversion Tab: - Landing page → Signup rate - Signup → Purchase rate - Purchase → Pass rate - Pass

→ Payout rate

Campaigns Tab: - Active campaigns - Performance by campaign - A/B test results

Reports Tab: - Export data for external analysis - Custom date ranges - Demographic breakdowns

Important: Marketing team has READ-ONLY access. Cannot modify anything.

7.6 Developer Dashboard

Route: /admin/dev

Access: Developer + Super Admin

Purpose: Monitor system health and deploy updates

Layout:

DEVELOPER DASHBOARD [Name] Logout	
SIDEBAR	MAIN CONTENT
System Logs Errors Deploy Database	SYSTEM STATUS API: ✔ Healthy Database: ✔ Connected MT5 API: ✔ Active Payment API: ✔ Active Monitoring Job: ✔ Running RECENT ERRORS (Last 24h) ⚠ Payment webhook timeout Count: 3 Last: 2 hours ago [View Details] [Fix] ⚠ MT5 login failed (User #234) Status: Resolved

Key Features:

System Health: - Uptime monitoring - API response times - Database connection status - External service status (Flutterwave, MT5, Email)

Logs Tab: - Real-time system logs - Filter by: Level (info, warning, error), Service, Date - Search functionality

Errors Tab: - Error tracking - Stack traces - Frequency analysis - Status: Open, In Progress, Resolved

Deploy Tab: - Recent deployments - Rollback option (if deployment breaks something) - Deploy new version (links to GitHub releases)

Database Tab: - Database stats (size, connections, queries) - Run manual queries (for debugging only) - Database backups list

Important: Developer has NO access to user personal data, financial data, or business operations.

8. ROLE-BASED ACCESS CONTROL (RBAC)

8.1 Implementation Requirements

CRITICAL: Every API endpoint must check permissions before executing.

Example middleware:

```
// Pseudocode

function requireRole(allowedRoles) {
  return async (req, res, next) => {
    const user = req.user; // From JWT

    if (!user) {
      return res.status(401).json({ error: 'Unauthorized' });
    }

    if (!allowedRoles.includes(user.role)) {
      return res.status(403).json({ error: 'Forbidden' });
    }

    next();
  };
}

// Usage:
app.get('/api/admin/revenue',
  requireAuth,
  requireRole(['super_admin']),
  getRevenueStats
);

app.put('/api/payouts/:id/approve',
  requireAuth,
  requireRole(['super_admin', 'compliance']),
  approvePayout
);
```

8.2 Frontend Route Protection

Routes must be protected based on role:

```
// Example in React Router

<Routes>
  { /* Public routes */ }
  <Route path="/" element={<Homepage />} />
  <Route path="/login" element={<Login />} />

  { /* Trader routes */ }
  <Route
    path="/dashboard/*"
    element={
      <ProtectedRoute allowedRoles={['trader']}>
        <TraderDashboard />
      </ProtectedRoute>
    }
  />

  { /* Admin routes */ }
  <Route
    path="/admin"
    element={
      <ProtectedRoute allowedRoles={['super_admin']}>
        <SuperAdminDashboard />
      </ProtectedRoute>
    }
  />

  <Route
    path="/admin/compliance"
    element={
      <ProtectedRoute allowedRoles={['super_admin', 'compliance']}>
        <ComplianceDashboard />
      </ProtectedRoute>
    }
  />

  { /* etc */ }
</Routes>
```

8.3 UI Element Hiding

Hide buttons/sections based on permissions:

```
// Example in React

function PayoutCard({ payout, currentUser }) {
  const canApprove = ['super_admin', 'compliance'].includes(currentUser.role);

  return (
    <div>
      <h3>Payout #{payout.id}</h3>
      <p>Amount: {payout.amount}</p>

      {canApprove && payout.status === 'pending' && (
        <div>
          <button onClick={() => approve(payout.id)}>Approve</button>
          <button onClick={() => reject(payout.id)}>Reject</button>
        </div>
      )}
    </div>
  );
}
```

9. PAYMENT INTEGRATION (FLUTTERWAVE)

9.1 Payment Flow

User clicks "Buy Challenge"
↓
Frontend calls POST /api/payments/initiate
↓
Backend creates transaction record (status: pending)
Backend calls Flutterwave API to generate payment link
↓
Frontend redirects user to Flutterwave payment page
↓
User enters card details and pays
↓
Flutterwave processes payment
↓
Flutterwave sends webhook to POST /api/payments/webhook
↓
Backend verifies webhook signature
Backend updates transaction status to 'completed'
Backend creates MT5 account via broker API
Backend sends email with MT5 credentials
↓
User redirected back to success page

9.2 Flutterwave Integration Endpoints

Initiate Payment:

```
POST /api/payments/initiate

Request Body:
{
  "challenge_type": "naira", // or "dollar"
  "amount": 10000,
  "currency": "NGN"
}

Response:
{
  "success": true,
  "data": {
    "transaction_id": "uuid",
    "payment_link": "https://checkout.flutterwave.com/...",
    "reference": "NF_12345"
  }
}
```

Webhook Handler:

```

POST /api/payments/webhook

Headers:
{
  "verif-hash": "flutterwave_webhook_signature"
}

Body: (from Flutterwave)
{
  "event": "charge.completed",
  "data": {
    "id": 12345,
    "tx_ref": "NF_12345",
    "amount": 10000,
    "currency": "NGN",
    "customer": {
      "email": "user@email.com"
    },
    "status": "successful"
  }
}

Backend Actions:
1. Verify webhook signature
2. Check transaction hasn't been processed already
3. Update transaction status
4. Create MT5 account
5. Send email with credentials
6. Return 200 OK to Flutterwave

```

Security Requirements:

1. **Verify Webhook Signature:**
 - Flutterwave sends verif-hash header
 - Compare with your webhook secret
 - Reject if doesn't match
2. **Idempotency:**
 - Flutterwave may send webhook multiple times
 - Check if transaction already processed
 - Don't create duplicate MT5 accounts
3. **Amount Verification:**
 - Verify paid amount matches expected amount
 - Don't trust client-side data

9.3 Supported Payment Methods

Via Flutterwave: - Nigerian bank cards (Verve, Visa, Mastercard) - Bank transfers - USSD - Bank accounts - Mobile money

Important: Flutterwave handles all payment processing. We just receive webhooks.

10. MT5 INTEGRATION

10.1 Broker API Requirements

We need the broker to provide:

1. Account Creation API

```

POST /api/create_account

Request:
{
  "group": "demo_challenge",
  "balance": 200000,
  "leverage": 100,
  "name": "John Doe",
  "email": "john@email.com"
}

Response:

```

```
{
  "login": "12345678",
  "password": "ABC123xyz",
  "server": "Broker-Demo"
}
```

2. Get Account Data API

GET /api/account/{login}

Response:

```
{
  "login": "12345678",
  "balance": 250000,
  "equity": 253000,
  "margin": 12000,
  "free_margin": 241000,
  "profit": 53000
}
```

3. Get Trade History API

GET /api/account/{login}/trades

Response:

```
{
  "trades": [
    {
      "ticket": 98765,
      "symbol": "EURUSD",
      "type": "buy",
      "volume": 1.0,
      "open_price": 1.1050,
      "close_price": 1.1080,
      "profit": 300,
      "open_time": "2026-02-20T10:30:00Z",
      "close_time": "2026-02-20T14:30:00Z"
    }
  ]
}
```

4. Disable Account API

POST /api/account/{login}/disable

Response:

```
{
  "success": true,
  "message": "Account disabled"
}
```

10.2 Integration Points

When payment succeeds: - Call broker API to create MT5 account - Store login, password, server in database - Email credentials to user

Every 60 seconds: - Call broker API for all active challenge accounts - Get current balance, equity - Check for rule violations - Disable account if violation detected

When user requests payout: - Compliance reviews - If approved, notify broker - Broker creates real funded account

10.3 Error Handling

If MT5 account creation fails: - Log error - Store transaction as "pending_account_creation" - Retry automatically every 5 minutes (max 10 attempts) - If still fails after 10 attempts, alert admin and refund user

If MT5 data fetch fails: - Log warning - Continue monitoring other accounts - Retry on next cycle - Alert admin if fails consistently for same account

11. REAL-TIME MONITORING SYSTEM

11.1 Architecture

Monitoring Service (Separate from API):

MAIN API (User requests)

MONITORING SERVICE (Background)

- Runs every 60 seconds
- Checks all active challenges
- Independent from main API
- If crashes, auto-restarts

11.2 Implementation Details

Using Node.js + node-cron:

```
const cron = require('node-cron');

// Run every minute
cron.schedule('* * * * *', async () => {
  console.log('Starting monitoring cycle...');

  try {
    await monitorAllChallenges();
  } catch (error) {
    console.error('Monitoring cycle failed:', error);
    // Alert admin
    await sendAlertToAdmin('Monitoring system error', error);
  }
});

async function monitorAllChallenges() {
  const activeChallenges = await db.query(
    'SELECT * FROM challenges WHERE status = $1',
    ['active']
  );

  console.log(`Monitoring ${activeChallenges.length} challenges`);

  // Process in batches to avoid overwhelming broker API
  const batchSize = 50;
  for (let i = 0; i < activeChallenges.length; i += batchSize) {
    const batch = activeChallenges.slice(i, i + batchSize);

    await Promise.all(
      batch.map(challenge => checkChallenge(challenge))
    );
  }
}

async function checkChallenge(challenge) {
  try {
    // Get live MT5 data
    const mt5Data = await brokerAPI.getAccountData(challenge.mt5_login);

    // Update current values
    await db.query(
      'UPDATE challenges SET current_balance = $1, current_equity = $2, updated_at = NOW() WHERE id = ' +
      [mt5Data.balance, mt5Data.equity, challenge.id]
    );

    // Check rules
    const violations = checkRules(challenge, mt5Data);

    if (violations.length > 0) {
      await handleViolation(challenge, violations[0], mt5Data);
    }

    // Check if passed
    if (hasPassedChallenge(challenge, mt5Data)) {
      await handlePass(challenge, mt5Data);
    }
  }
}
```

```

    }

    } catch (error) {
      console.error(`Error checking challenge ${challenge.id}:`, error);
      // Don't throw - continue with other challenges
    }
  }
}

```

11.3 Alerting System

Send alerts when: - Monitoring system stops running - MT5 API is down for > 5 minutes - Database connection lost - More than 10% of checks failing - Any critical error

Alert channels: - Email to admin - SMS to admin (Twilio) - Slack/Discord webhook (optional)

11.4 Performance Requirements

System must: - Handle 1000+ concurrent challenges - Complete one monitoring cycle in < 60 seconds - Use < 1GB RAM - Auto-recover from crashes - Keep running even if main API is down

12. SECURITY REQUIREMENTS

12.1 Authentication & Authorization

Password Requirements: - Minimum 8 characters - Must contain: uppercase, lowercase, number - Hashed using bcrypt (cost factor: 12)

JWT Tokens: - Access token: 15 minutes expiry - Refresh token: 7 days expiry - Stored in httpOnly cookies (not localStorage) - Include: user_id, email, role

Two-Factor Authentication (Phase 2): - TOTP (Time-based One-Time Password) - Using authenticator apps (Google Authenticator, Authy)

12.2 API Security

Rate Limiting: - 100 requests per minute per IP for public endpoints - 1000 requests per minute for authenticated users - 10 requests per minute for login endpoint (prevent brute force)

CORS: - Allow only from: noblefunded.com, www.noblefunded.com - No wildcard (*) origins

Security Headers:

```

X-Frame-Options: DENY
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=31536000
Content-Security-Policy: default-src 'self'

```

12.3 Data Protection

Sensitive Data Encryption: - MT5 passwords encrypted at rest - Payout bank details encrypted - API keys stored in environment variables

Personal Data: - GDPR/NDPR compliant - Users can request data export - Users can request account deletion

Database: - SSL connection required - Automated daily backups - Backups stored encrypted

12.4 Input Validation

All user inputs must be validated: - Email: Valid format - Phone: Valid Nigerian format - Amounts: Positive numbers only - Text fields: Max length, no SQL injection - File uploads: Max 5MB, allowed types only

Never trust client-side validation alone. Always validate on backend.

12.5 Audit Logging

Log all sensitive actions: - Login attempts (success and failure) - Password changes - Payout approvals/rejections - User bans - Team member additions/removals - System setting changes

Logs must include: - Timestamp - User who performed action - Action type - Resource affected - IP address - Result (success/failure)

13. DEPLOYMENT & INFRASTRUCTURE

13.1 Hosting Setup

Frontend (Netlify): - Connect to GitHub repo - Auto-deploy on push to main branch - Build command: `npm run build` - Publish directory: `dist` or `build` - Environment variables: `API_URL`

Backend (Render): - Web Service for API - Background Worker for monitoring cron job - Connect to GitHub repo - Auto-deploy on push to main branch - Build command: `npm install` - Start command: `npm start` - Environment variables: (see below)

Database (Render PostgreSQL): - Managed PostgreSQL instance - Automatic backups - Connection URL provided by Render

13.2 Environment Variables

Backend requires:

```
# Database
DATABASE_URL=postgresql://...

# JWT
JWT_SECRET=random_secret_key_here
JWT_REFRESH_SECRET=another_secret_here

# Flutterwave
FLUTTERWAVE_PUBLIC_KEY=FLWPUBK-...
FLUTTERWAVE_SECRET_KEY=FLWSECK-...
FLUTTERWAVE_WEBHOOK_SECRET=...

# Broker MT5 API
MT5_API_URL=https://broker-api.com
MT5_API_KEY=...

# Email (SendGrid)
SENDGRID_API_KEY=SG....
FROM_EMAIL=noreply@noblefunded.com

# Frontend URL
FRONTEND_URL=https://noblefunded.com

# Admin Email (for alerts)
ADMIN_EMAIL=admin@noblefunded.com

# Node Environment
NODE_ENV=production
```

13.3 Domain Setup

DNS Records (at Namecheap or registrar):

Type	Name	Value	TTL
A	@	[Netlify IP]	Automatic
CNAME	www	[Netlify hostname]	Automatic
CNAME	api	[Render hostname]	Automatic

SSL Certificates: - Netlify provides automatic SSL - Render provides automatic SSL - No manual certificate management needed

13.4 Monitoring & Logging

Setup monitoring tools:

Uptime Monitoring: - UptimeRobot (free) - Check main site every 5 minutes - Alert if down

Error Tracking: - Sentry (free tier: 5K errors/month) - Captures all application errors - Sends alerts on new errors

Logs: - Render provides log streaming - Keep last 7 days of logs

13.5 Backup Strategy

Database Backups: - Automated daily backups (Render built-in) - Manual backup before major updates - Keep backups for 30 days

Code Backups: - GitHub is primary backup - All code pushed daily - Tagged releases for each deployment

14. TESTING REQUIREMENTS

14.1 Unit Tests

Required for: - Rule checking logic - Payment processing - Authentication functions - Database queries

Testing framework: Jest (JavaScript) or pytest (Python)

Minimum coverage: 70% code coverage

14.2 Integration Tests

Test scenarios: - Complete user registration flow - Complete payment and challenge creation flow - Rule violation detection - Payout approval flow - MT5 account creation

14.3 Manual Testing Checklist

Before launch, manually test:

User Flow: ☐ Register new account ☐ Login ☐ Buy challenge (test payment) ☐ Receive MT5 credentials email ☐ Login to dashboard ☐ See challenge details ☐ Request payout (after passing)

Admin Flow: ☐ Login as super admin ☐ View all users ☐ View all challenges ☐ Approve a payout ☐ Ban a user ☐ Add team member ☐ Change system settings

Security: ☐ Try accessing admin pages without login → should redirect ☐ Try accessing admin pages as regular user → should show forbidden ☐ Try SQL injection in forms → should be blocked ☐ Try XSS attacks → should be sanitized

Performance: ☐ Create 100 test challenges ☐ Verify monitoring completes in < 60 seconds ☐ Check database query performance

15. TIMELINE & MILESTONES

15.1 Development Phases

Phase 1: MVP (Weeks 1-8)

Week 1-2: Foundation - Database schema setup - Basic authentication (register, login) - GitHub repo setup - Development environment

Week 3-4: Core Features - Payment integration (Flutterwave) - MT5 account creation - Challenge purchase flow - Email notifications

Week 5-6: Dashboard & Monitoring - Trader dashboard - Real-time MT5 data sync - Rule monitoring system - Challenge status updates

Week 7-8: Admin Panel - Super admin dashboard - User management - Challenge management - Basic payout system

Deliverable: Working MVP with all core features

Phase 2: Polish & Launch (Weeks 9-10)

Week 9: - Bug fixes - Security hardening - Performance optimization - Testing (unit + integration)

Week 10: - Production deployment - Final testing - Soft launch to first 50 users - Monitor and fix issues

Deliverable: Production-ready platform

Phase 3: Post-Launch (Weeks 11-12)

Week 11-12: - Role-based admin dashboards (Compliance, Support, Marketing) - Payout approval workflow - Advanced reporting - Activity logging

Deliverable: Complete admin system

Phase 4: Enhancements (Month 4+)

Future features (not part of initial build): - Mobile app (React Native) - Social features (leaderboards, challenges) - Referral system - Advanced analytics - Copy trading - Multiple broker integrations

15.2 Milestone Checklist

Milestone 1: Database + Auth (Week 2) ☐ Database schema created ☐ Registration works ☐ Login works ☐ JWT authentication implemented ☐ Password reset flow works

Milestone 2: Payment + MT5 (Week 4) ☐ Flutterwave integration complete ☐ Payment webhook works ☐ MT5 account created after payment ☐ Credentials emailed to user

Milestone 3: Monitoring (Week 6) ☐ MT5 data sync working ☐ Rule checking logic complete ☐ Accounts auto-disabled on violation ☐ Monitoring runs every 60 seconds

Milestone 4: Dashboards (Week 8) ☐ Trader dashboard complete ☐ Admin dashboard complete ☐ All CRUD operations work ☐ Real-time data displays correctly

Milestone 5: Launch Ready (Week 10) ☐ All tests passing ☐ Security review complete ☐ Deployed to production ☐ Domain connected ☐ First test user completes full flow

16. DELIVERABLES

16.1 What Developer Must Provide

At end of project:

1. **Complete Source Code**
 - All code in GitHub repository
 - Well-commented
 - README with setup instructions
2. **Deployment Documentation**
 - How to deploy updates
 - Environment variables list
 - Troubleshooting guide
3. **API Documentation**
 - All endpoints documented
 - Request/response examples
 - Error codes explained
4. **Admin Guide**
 - How to use each admin dashboard
 - Common tasks (add user, approve payout, etc.)
 - Screenshots
5. **Database Migrations**
 - All migration scripts
 - How to run migrations
 - Rollback procedures
6. **Test Accounts**

- Super admin account
- Test trader account
- Sample data for testing

16.2 Ongoing Support

After launch, developer provides: - Bug fixes (priority within 24 hours) - Feature enhancements (as agreed) - Performance optimization - Security updates - Monthly maintenance (10-15 hours)

17. COMMUNICATION & COLLABORATION

17.1 Development Workflow

Daily: - Developer commits code to GitHub - Push to develop branch (not main)

Weekly: - Progress meeting (1 hour) - Demo of completed features - Review next week's tasks

Code Review: - Pull requests required before merging to main - Founder (or designated reviewer) approves - Automated tests must pass

17.2 Project Management

Use Trello or Linear for tasks: - Columns: To Do, In Progress, Review, Done - Each feature is a card - Developer moves cards as progresses

Documentation: - Developer maintains changelog - Documents major decisions - Updates README as needed

18. ACCEPTANCE CRITERIA

18.1 Definition of Done

A feature is "done" when: ☐ Code is written and committed ☐ Unit tests pass ☐ Manually tested and works ☐ Deployed to staging ☐ Approved by founder ☐ Documentation updated

18.2 Launch Criteria

Platform is ready to launch when: ☐ All Phase 1 features complete ☐ All security requirements met ☐ All tests passing (100% of critical paths) ☐ Performance requirements met ☐ At least 3 complete end-to-end test flows successful ☐ Error tracking setup ☐ Backups configured ☐ Founder can successfully complete admin tasks

19. QUESTIONS FOR DEVELOPER

Before starting, developer should answer:

- 1. Technical Approach:**
 - What tech stack do you recommend?
 - How will you structure the codebase?
 - What database migrations tool will you use?
- 2. Timeline:**
 - Do you agree with the 8-10 week timeline?
 - What are potential blockers?
 - When can you start?
- 3. Monitoring:**
 - How will you implement the 24/7 monitoring?
 - What happens if monitoring crashes?
 - How will you handle 1000+ concurrent challenges?
- 4. Security:**
 - What security measures will you implement?
 - How will you prevent common vulnerabilities?
 - Experience with PCI DSS for payment handling?
- 5. Deployment:**

- Comfortable with Netlify + Render?
 - How will you handle deployments?
 - What's your rollback strategy?
-

20. APPENDIX

20.1 Glossary

- **Challenge:** A evaluation period where trader proves profitability
- **MT5:** MetaTrader 5, trading platform
- **Drawdown:** Loss from peak balance
- **Equity:** Current account value including open positions
- **Payout:** Payment to trader after passing challenge
- **Violation:** Breaking challenge rules (triggers failure)

20.2 Reference Materials

Share with developer: - FTMO (ftmo.com) - Example prop firm to study - FundedNext (fundednext.com) - Example prop firm to study - Flutterwave API Docs (developer.flutterwave.com) - This technical requirements document

20.3 Contact Information

Founder: [Your Name] **Email:** contact@noblefunded.com **Phone/WhatsApp:** [Your Number]

Preferred communication: - Quick questions: WhatsApp - Feature discussions: Video call - Bug reports: GitHub Issues - Weekly updates: Zoom/Google Meet

SIGNATURE

Project Owner: Name: _____ Signature: _____ Date: _____

Developer: Name: _____ Signature: _____ Date: _____

END OF DOCUMENT

Total Pages: [To be determined after formatting] Version: 1.0 Last Updated: February 20, 2026